NAME: K. SWARNA VARSHINI

REG.NO: 192324081

COURSE CODE: CSA0920

GITHUB LINK:

https://github.com/kswarna1/CSA0920-JAVA.git

Q1– Handling Failure in a Bank ATM Simulation

Aim:

To implement an ATM withdrawal system using exception handling concepts such as try, catch, multi-catch, finally, and user-defined exceptions.

Theory:

Exception handling manages runtime problems without terminating the program unexpectedly.

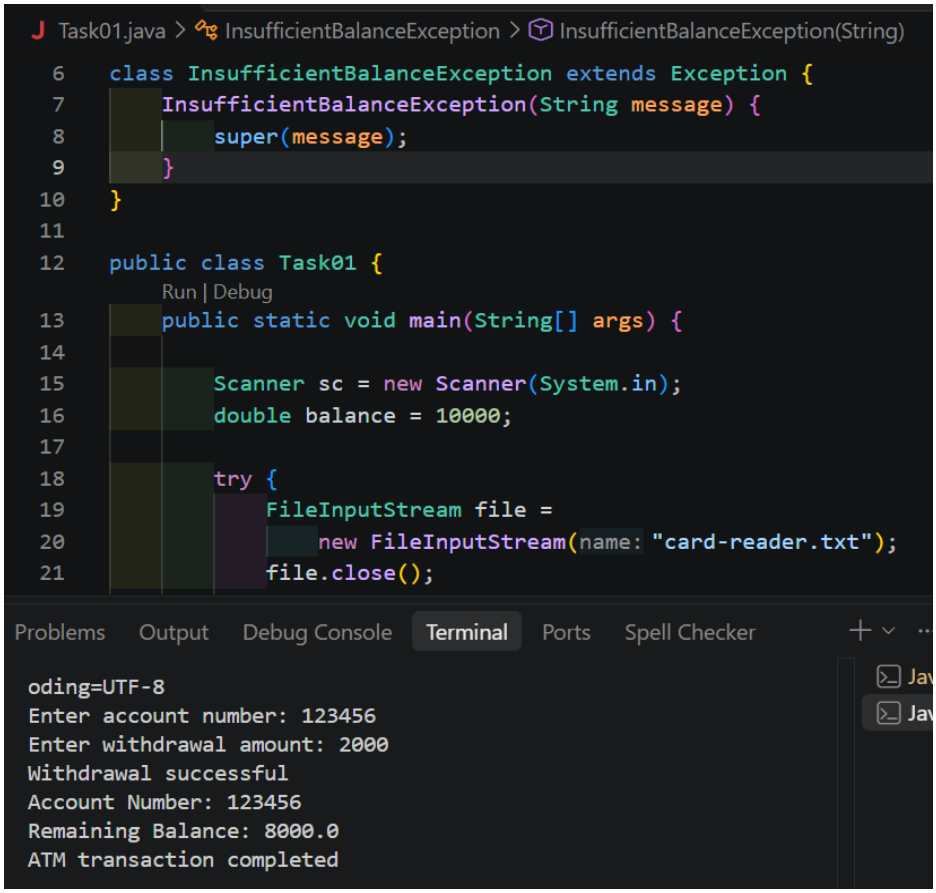
- try: Contains code that may produce an exception.
- catch: Handles the exception.
- finally: Executes whether an exception occurs or not.
- throw: Explicitly throws an exception.
- throws: Declares an exception that a method may pass to its caller.
- Checked exceptions must be caught or declared.
- Unchecked exceptions occur during runtime.

Java Program:

```
import java.util.Scanner;
import java.io.FileInputStream;
import java.io.IOException;
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}
public class Task01 {
    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
double balance = 10000;
try {
    FileInputStream file =
        new FileInputStream("card-reader.txt");
    file.close();
    System.out.print("Enter account number: ");
    String account = sc.nextLine();
    System.out.print("Enter withdrawal amount: ");
    double amount = Double.parseDouble(sc.nextLine());
    if (amount <= 0) {
        throw new IllegalArgumentException(
            "Amount must be positive");
    }
    if (amount > balance) {
        throw new InsufficientBalanceException(
            "Insufficient account balance");
    }
    balance = balance - amount;
    System.out.println("Withdrawal successful");
    System.out.println("Account Number: " + account);
    System.out.println("Remaining Balance: " + balance);
} catch (IOException e) {
    System.out.println("Card reader file is missing");
```

```
        } catch (IllegalArgumentException e) {
            System.out.println("Invalid withdrawal amount: "
                    + e.getMessage());
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("ATM transaction completed");
            sc.close();
        }
    }
}
```



Q2– User-Defined Exceptions and Propagation in an Online Examination Portal

Aim

To create user-defined checked exceptions and demonstrate exception propagation and exception chaining in an online examination portal.

Theory

User-defined exceptions are created by extending the Exception or RuntimeException class.

- throw is used to throw an exception.
- throws declares exceptions in a method signature.
- Exception propagation passes an exception from one method to its caller.
- Exception chaining preserves the original cause using getCause().

Java Program

```
import java.io.IOException;

class ExamException extends Exception {
    ExamException(String message) {
        super(message);
    }
}

class SubmissionStorageException extends ExamException {
    SubmissionStorageException(String message, Throwable cause) {
        super(message);
        initCause(cause);
    }
}

class Storage {
```

```
    void save(String fileName, int fileSize)
            throws SubmissionStorageException {
        try {
            if (fileSize > 500) {
                throw new IOException("Disk quota exceeded");
            }
            System.out.println("Saved " + fileName);
        } catch (IOException e) {
            throw new SubmissionStorageException(
                "Could not store " + fileName, e);
        }
    }
}
public class Task02 {
    public static void main(String[] args) {
        Storage storage = new Storage();
        try {
            storage.save("answer01.pdf", 120);
            storage.save("answer02.pdf", 800);
        } catch (ExamException e) {
            System.out.println("Error: " + e.getMessage());
            if (e.getCause() != null) {
                System.out.println("Cause: "
                                   + e.getCause().getMessage());
```

```
    }
  }
}
```

```
33  public class Task02 {
34
        Run | Debug
35      public static void main(String[] args) {
36
37          Storage storage = new Storage();
38
39          try {
40              storage.save(fileName: "answer01.pdf", fileSize: 120);
41              storage.save(fileName: "answer02.pdf", fileSize: 800);
42
43          } catch (ExamException e) {
44
45              System.out.println("Error: " + e.getMessage());
46
47              if (e.getCause() != null) {
48                  System.out.println("Cause: "
49                                     + e.getCause().getMessage());
```

Problems Output Debug Console **Terminal** Ports Spell Checker

```
D:\JAVA\CSA09-192324081-Assignment3>java Task02
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.enc
oding=UTF-8
Saved answer01.pdf
Error: Could not store answer02.pdf
Cause: Disk quota exceeded
```

Q3- Resource Safety, Debugging and Robust Code

Aim

To develop a Java program to read sales data from multiple store files, calculate the total sales, and generate a summary report using exception handling and try-with-resources to ensure proper resource management.

Theory

Resource management is important in Java when working with files, databases, and network connections.

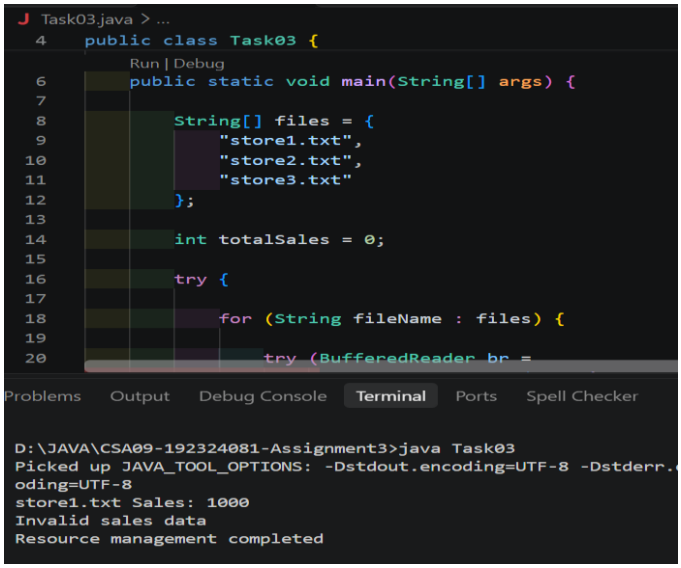
- File Handling: Used to read and write data from files.
- Exception Handling: Handles errors using try, catch, and finally.

- Try-with-resources: Automatically closes files after use.
- IOException: Handles file-related errors.
- NumberFormatException: Handles invalid numeric data.
- Debugging: Helps identify and correct errors in a program.

Java Program

```
import java.io.*;
public class Task03 {
    public static void main(String[] args) {
        String[] files = {
            "store1.txt",
            "store2.txt",
            "store3.txt"
        };
        int totalSales = 0;
        try {
            for (String fileName : files) {
                try (BufferedReader br =
                     new BufferedReader(new FileReader(fileName))) {
                    String data = br.readLine();
                    int sales = Integer.parseInt(data);
                    totalSales += sales;
                    System.out.println(fileName +
                                       " Sales: " + sales);
                }
```

```
        }
        try (BufferedWriter bw =
            new BufferedWriter(
                new FileWriter("summary.txt"))) {
            bw.write("Total Sales: " + totalSales);
        }
        System.out.println("Summary report generated");
        System.out.println("Total Sales: " + totalSales);
    } catch (IOException e) {
        System.out.println("File error: " + e.getMessage());
    } catch (NumberFormatException e) {
        System.out.println("Invalid sales data");
    } finally {
        System.out.println("Resource management completed");
    }
  }
}
```



Q4– Thread Creation and the Thread Life Cycle

Aim

To implement a Java program using multithreading to simulate four scanner stations processing scripts simultaneously using the Thread class, Runnable interface, and thread methods.

Theory

Multithreading is a Java feature that allows multiple threads to execute concurrently within a program.

Important concepts:

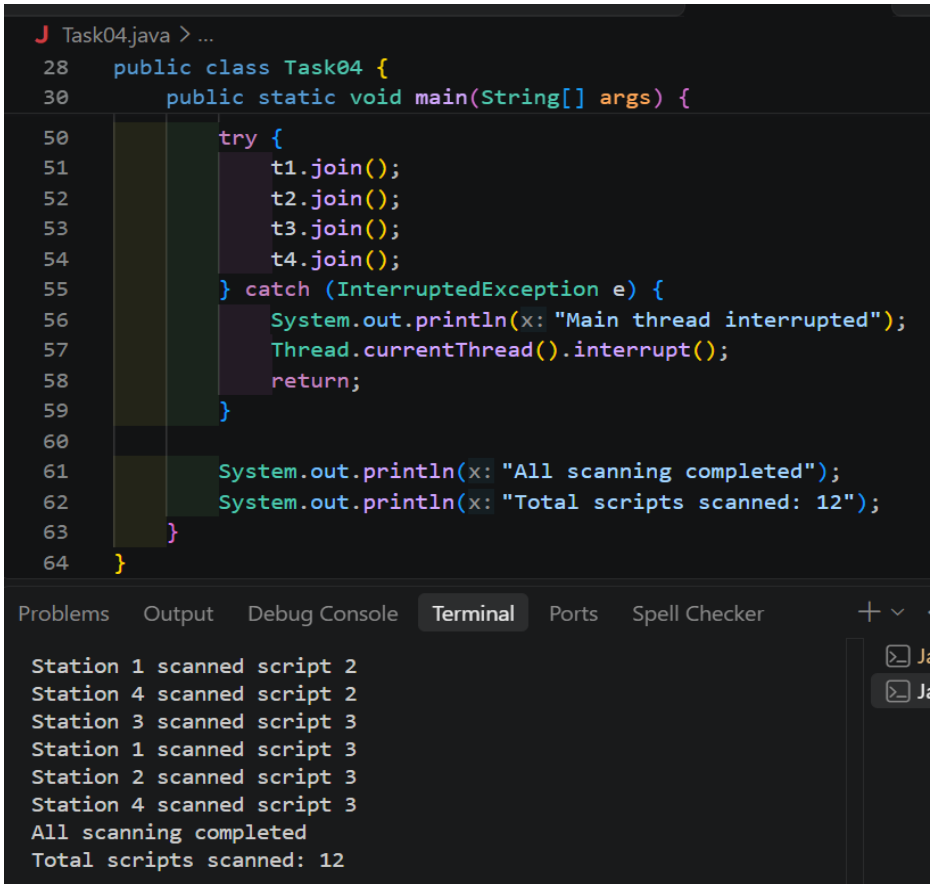
1. Thread: A small unit of execution within a program.
2. Runnable Interface: Used to create threads by implementing the run() method.
3. start(): Starts a new thread.
4. run(): Contains the task performed by a thread.
5. sleep(): Pauses a thread temporarily.
6. join(): Waits for a thread to finish.
7. isAlive(): Checks whether a thread is still running.
8. Thread Priority: Determines the relative scheduling priority.
9. Daemon Thread: A background thread that does not prevent JVM shutdown.

Java Program

```
class ScannerStation implements Runnable {
    private String stationName;
    ScannerStation(String name) {
        stationName = name;
    }
    public void run() {
        for (int i = 1; i <= 3; i++) {
```

```
            System.out.println(stationName +
                               " scanned script " + i);
            try {
                Thread.sleep(200);
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted");
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}
public class Task04 {
    public static void main(String[] args) {
        ScannerStation s1 = new ScannerStation("Station 1");
        ScannerStation s2 = new ScannerStation("Station 2");
        ScannerStation s3 = new ScannerStation("Station 3");
        ScannerStation s4 = new ScannerStation("Station 4");
        Thread t1 = new Thread(s1);
        Thread t2 = new Thread(s2);
        Thread t3 = new Thread(s3);
        Thread t4 = new Thread(s4);
        t1.start();
        t2.start();
```

```
        t3.start();
        t4.start();
        System.out.println("Station 1 is alive: " +
                           t1.isAlive());
        try {
            t1.join();
            t2.join();
            t3.join();
            t4.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted");
            Thread.currentThread().interrupt();
            return;
        }
        System.out.println("All scanning completed");
        System.out.println("Total scripts scanned: 12");
    }
}
```



Q5– Race Conditions in a Cinema Seat-Booking System

Aim

To develop a Java program to demonstrate race conditions in a cinema ticket booking system and prevent multiple threads from booking the same seat using synchronization.

Theory

Multithreading allows multiple threads to execute simultaneously. When multiple threads access and modify shared data at the same time, a race condition may occur.

Important concepts:

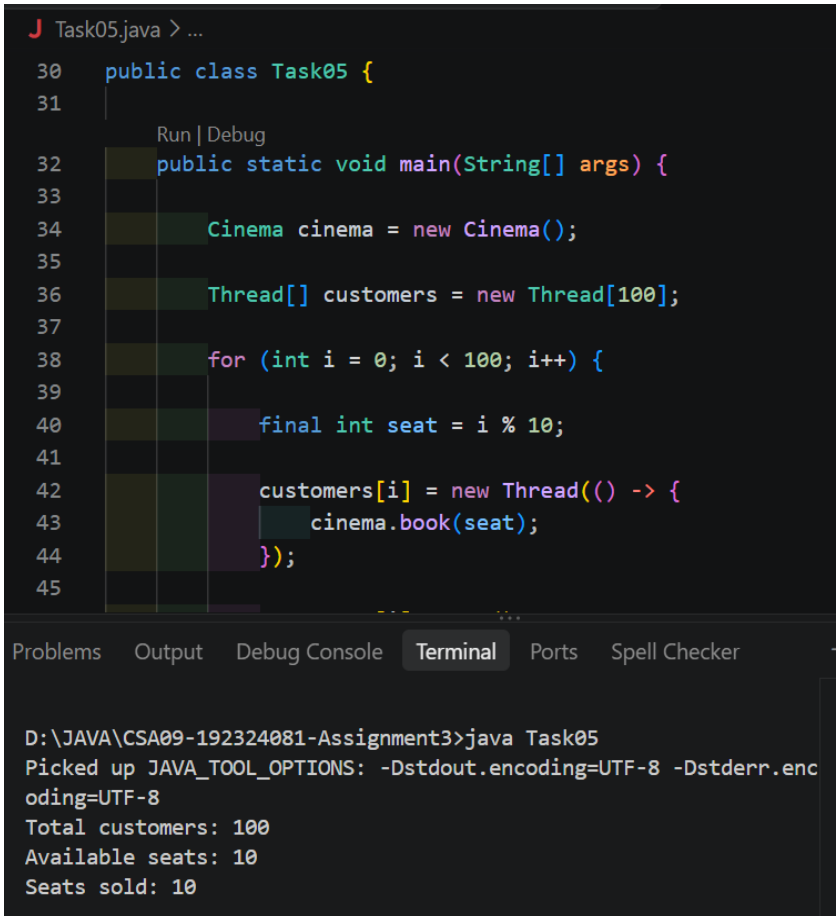
1. Race Condition: Occurs when multiple threads access shared data simultaneously and produce incorrect results.
2. Synchronization: Controls access to shared resources.
3. Synchronized Method: Allows only one thread at a time to execute the synchronized method on the same object.

4. Synchronized Block: Protects a specific section of code.
5. Thread: Represents an independent execution path.
6. Lock: Prevents multiple threads from accessing a protected section simultaneously.

Java Program

```
class Cinema {
    boolean[] seats = new boolean[10];
    synchronized void book(int seat) {
        if (!seats[seat]) {
            seats[seat] = true;
        }
    }
    int getTotalSeatsSold() {
        int count = 0;
        for (boolean seat : seats) {
            if (seat) {
                count++;
            }
        }
        return count;
    }
}
public class Task05 {
    public static void main(String[] args) {
```

```
        Cinema cinema = new Cinema();
        Thread[] customers = new Thread[100];
        for (int i = 0; i < 100; i++) {
            final int seat = i % 10;
            customers[i] = new Thread(() -> {
                cinema.book(seat);
            });
            customers[i].start();
        }
        try {
            for (Thread t : customers) {
                t.join();
            }
        } catch (InterruptedException e) {
            System.out.println("Booking interrupted");
            Thread.currentThread().interrupt();
            return;
        }
        System.out.println("Total customers: 100");
        System.out.println("Available seats: 10");
        System.out.println("Seats sold: " +
                           cinema.getTotalSeatsSold());
    }
}
```



Q6– Deadlock and Inter-Thread Communication

Aim

To develop a Java program to demonstrate deadlock prevention using lock ordering and implement the producer-consumer problem using inter-thread communication methods such as wait() and notifyAll().

Theory

Deadlock: A situation where two or more threads wait indefinitely for resources held by each other.

Deadlock Prevention: Lock ordering ensures that threads acquire shared resources in a consistent order, reducing the possibility of deadlocks.

Producer-Consumer Problem: A multithreading problem in which one thread produces data and another thread consumes it through a shared buffer.

Java Program

```
class Wallet {
    int id;
    int balance;
    Wallet(int id, int balance) {
        this.id = id;
        this.balance = balance;
    }
    static void transfer(Wallet a, Wallet b, int amount) {
        Wallet first = a.id < b.id ? a : b;
        Wallet second = a.id < b.id ? b : a;
        synchronized (first) {
            synchronized (second) {
                a.balance -= amount;
                b.balance += amount;
            }
        }
    }
}
class TokenCounter {
    private int[] buffer = new int[5];
    private int front = 0;
    private int rear = 0;
    private int count = 0;
```

```
    synchronized void put(int value)
            throws InterruptedException {
        while (count == buffer.length) {
            wait();
        }
        buffer[rear] = value;
        rear = (rear + 1) % buffer.length;
        count++;
        notifyAll();
    }
    synchronized int take()
            throws InterruptedException {
        while (count == 0) {
            wait();
        }
        int value = buffer[front];
        front = (front + 1) % buffer.length;
        count--;
        notifyAll();
        return value;
    }
}
public class Task06 {
```

```
public static void main(String[] args) {
    Wallet w1 = new Wallet(1, 1000);
    Wallet w2 = new Wallet(2, 1000);
    Thread t1 = new Thread(() ->
        Wallet.transfer(w1, w2, 100));
    Thread t2 = new Thread(() ->
        Wallet.transfer(w2, w1, 200));
    t1.start();
    t2.start();
    try {
        t1.join();
        t2.join();
    } catch (InterruptedException e) {
        System.out.println("Transaction interrupted");
        Thread.currentThread().interrupt();
        return;
    }
    System.out.println("Wallet 1 Balance: " + w1.balance);
    System.out.println("Wallet 2 Balance: " + w2.balance);
    TokenCounter buffer = new TokenCounter();
    final int[] sum = {0};
    Thread producer = new Thread(() -> {
        try {
            for (int i = 1; i <= 20; i++) {
```

```
            buffer.put(i);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});
Thread consumer = new Thread(() -> {
    try {
        for (int i = 1; i <= 20; i++) {
            sum[0] += buffer.take();
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});
producer.start();
consumer.start();
try {
    producer.join();
    consumer.join();
} catch (InterruptedException e) {
    System.out.println("Processing interrupted");
    Thread.currentThread().interrupt();
    return;
```

```
        }
        System.out.println("Producer-Consumer Sum: " + sum[0]);
        System.out.println("Processing completed");
    }
}
```

```
66  public class Task06 {
68      public static void main(String[] args) {
110             try {
111                 for (int i = 1; i <= 20; i++) {
112                     sum[0] += buffer.take();
113                 }
114             } catch (InterruptedException e) {
115                 Thread.currentThread().interrupt();
116             }
117         });
118
119         producer.start();
120         consumer.start();
121
122         try {
123             producer.join();
124             consumer.join();

Problems  Output  Debug Console  Terminal  Ports  Spell Checker

D:\JAVA\CSA09-192324081-Assignment3>java Task06
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.e
oding=UTF-8
Wallet 1 Balance: 1100
Wallet 2 Balance: 900
Producer-Consumer Sum: 210
Processing completed
```

Q7– Design a Concurrent, Fault-Tolerant Processor from First Principles

Aim

To develop a Java program to process 10,000 answer files concurrently using ExecutorService, Callable, Future, and concurrent collections while handling missing and corrupted files without stopping the entire processing system.

Theory

Concurrent processing allows multiple tasks to execute at the same time, improving performance.

Important concepts:

1. ExecutorService: Manages a group of worker threads.
2. Callable: Executes a task and returns a result.
3. Future: Stores the result of an asynchronous task.
4. Concurrent Collections: Allow multiple threads to access shared data safely.
5. Fault Tolerance: Allows the application to continue processing even when some tasks fail.
6. ExecutionException: Handles exceptions that occur during task execution.
7. Thread Pool: Reuses a fixed number of threads to execute tasks.

Java Program

```
import java.util.*;
import java.util.concurrent.*;
import java.io.*;
public class Task07 {
    static class AnswerTask implements Callable<Integer> {
        private int fileNumber;
        AnswerTask(int fileNumber) {
            this.fileNumber = fileNumber;
        }
        public Integer call() throws Exception {
            if (fileNumber % 100 == 0) {
                throw new FileNotFoundException(
                    "Answer file missing");
            }
            if (fileNumber % 100 == 1) {
                throw new IOException(
```

```
                "Answer file corrupted");
        }
        return 1;
    }
}
static void processFiles(int poolSize)
        throws InterruptedException {
    ExecutorService executor =
        Executors.newFixedThreadPool(poolSize);
    ConcurrentLinkedQueue<Integer> successful =
        new ConcurrentLinkedQueue<>();
    ConcurrentLinkedQueue<Integer> missing =
        new ConcurrentLinkedQueue<>();
    ConcurrentLinkedQueue<Integer> corrupted =
        new ConcurrentLinkedQueue<>();
    List<Future<Integer>> results = new ArrayList<>();
    long startTime = System.currentTimeMillis();
    try {
        for (int i = 1; i <= 10000; i++) {
            results.add(executor.submit(
                new AnswerTask(i)));
        }
        for (Future<Integer> future : results) {
            try {
```

```
            future.get();
            successful.add(1);
        } catch (ExecutionException e) {
            Throwable cause = e.getCause();
            if (cause instanceof FileNotFoundException) {
                missing.add(1);
            } else {
                corrupted.add(1);
            }
        }
    }
} finally {
    executor.shutdown();
}
long endTime = System.currentTimeMillis();
System.out.println("Thread Pool Size: " + poolSize);
System.out.println("Successful Files: " +
                   successful.size());
System.out.println("Missing Files: " +
                   missing.size());
System.out.println("Corrupted Files: " +
                   corrupted.size());
System.out.println("Total Processed: " +
    (successful.size() + missing.size()
```

```
            + corrupted.size()));
        System.out.println("Time Taken: " +
                           (endTime - startTime) + " ms");
        System.out.println("--------------------------------");
    }
    public static void main(String[] args) {
        int[] poolSizes = {1, 2, 4, 8};
        try {
            for (int size : poolSizes) {
                processFiles(size);
            }
            System.out.println("All processing completed");
        } catch (InterruptedException e) {
            System.out.println("Processing interrupted");
            Thread.currentThread().interrupt();
        }
    }
}
```

